Redes Neuronales de Cuaterniones para Compresión de Imágenes

Universidad de O'Higgins

Andrés Zuñiga [1] Juan I. Riquelme [2]

Instituto de Ciencias de la ingeniería

Introducción

El Aprendizaje Automático requiere gran poder de cómputo para entrenar modelos, pero recursos mínimos para su ejecución. En la compresión de imágenes, entrenar una Red Neuronal de Cuaterniones (QNN) tiene un alto costo inicial, pero resulta en un modelo sumamente ligero capaz de procesar nuevos datos en tiempo real. Al tratar los canales RGB como una única entidad matemática, los *cuaterniones* preservan la estabilidad cromática y reducen la complejidad computacional en contraste con las redes tradicionales.

Objetivos de Investigación

El presente estudio, presentaremos los siguientes objetivos:

1. Implementar el Algoritmo Hebbiano Generalizado (QGHA) para entrenar Redes Neuronales de Cuaterniones orientadas al Análisis de Componentes Principales (QNN-PCA).
2. Evaluar la eficiencia de este modelo hipercomplejo en la reducción de dimensionalidad y la compresión de imágenes a color.

Cuaterniones

Los cuaterniones son una extensión de los números complejos en 4 dimensiones. Estos se definen como:

$$Q = a\mathbf{1} + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} \tag{1}$$

donde $\{a, b, c, d\} \in \mathbb{R}$, y $\{1, i, j, k\}$ forman la base del espacio de cuaterniones.

El producto escalar de dos cuaterniones Q_1, Q_2 se define como:

$$\langle Q_1, Q_2 \rangle = a_1a_2 + b_1b_2 + c_1c_2 + d_1d_2 \tag{2}$$

Luego, el vector normalizado $\hat{\mathbf{Q}}$ (también llamado *cuaternión unitario*) se calcula como:

$$\hat{\mathbf{Q}} = \frac{Q}{\|Q\|} = \frac{Q}{\langle Q, Q \rangle} = \frac{Q}{\sqrt{a^2 + b^2 + c^2 + d^2}} \tag{3}$$

Además definimos a el *producto de Hamilton* $\otimes$ de dos cuaterniones Q_1, Q_2, como:

$$\begin{aligned} Q_1 \otimes Q_2 = (a_1a_2 - b_1b_2 - c_1c_2 - d_1d_2) + (a_1b_2 + b_1a_2 + c_1d_2 - d_1c_2)\mathbf{i} \\ + (a_1c_2 - b_1d_2 + c_1a_2 + d_1b_2)\mathbf{j} + (a_1d_2 + b_1c_2 - c_1b_2 + d_1a_2)\mathbf{k} \end{aligned} \tag{4}$$

Implementación de QSVD

Sea $A \in Q^{n\times n}, rango(A) = m$, con $0 \leq m \leq n$, existen dos matrices unitarias generales de cuaterniones U y V tal que:

$$U^H AV = \begin{pmatrix} \Lambda_r & 0 \\ 0 & 0 \end{pmatrix}_{n\times n} \tag{5}$$

Donde H denota el operador de conjugada-transpuesta, y $\Lambda_r = diag\{\lambda_1, \ldots, \lambda_m\}$ y λ son valores singulares reales de A, las dos matrices unitarias U y V satisfacen $UU^H = VV^H = I_r$. Entonces podemos reescribir (5) como:

$$A = U \begin{pmatrix} \Lambda_r & 0 \\ 0 & 0 \end{pmatrix}_{n\times n} V^H \tag{6}$$

La aplicación de algunas propiedades a la QSVD (Descomposición de Valor Singular de Cuaterniones) es muy importante para la imagen en color, esto se resume de la siguiente manera:

- Invarianza a la rotación y desplazamiento espacial (también cierto en el caso de imágenes en escala de grises con SVD).
- Invarianza a la rotación del espacio de color.

Para una imagen que se va a comprimir, esta puede representarse como una matriz de cuaterniones $A \in Q^{n\times n}$; entonces, cada píxel en la imagen puede denotarse como un cuaternión imaginario puro, por lo que se puede emplear QSVD para descomponer la imagen:

$$A = U\Lambda V^H \tag{7}$$

Donde la fórmula es similar a la SVD de valor real; indica que una imagen en color se puede descomponer en la suma del producto exterior de vectores:

$$A = U\Lambda V^H = \sum_{i=1}^{m} \lambda_i (u_i \times v_i^H) \tag{8}$$

Donde u_i y v_i son vectores columna de la matriz U y V, respectivamente. Podemos definir cada $u_i \times v_i^H$ como una *eigen-imagen*, λ_i es el valor singular real en la matriz diagonal Λ. Dado que m es el rango de A, la información completa de una imagen en color se puede describir como la composición de m eigen-imágenes de color. Después de la QSVD, podemos elegir los primeros K Componentes Principales (PCs) para representar la imagen original, la imagen reconstruida se puede ver como:

$$\hat{A} = \sum_{i=1}^{K} \lambda_i (u_i \times v_i^H) \tag{9}$$

Red Neuronal de Cuaterniones

Dado un conjunto de vectores $X = (x_1, x_2, \ldots, x_N)$, donde $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T \in Q^{p\times 1}, i = 1, 2, \ldots, N$. Las muestras pueden someterse a una transferencia de componentes principales como:

$$Y = W^H X \tag{10}$$

Donde $W \in Q^{p\times j}(j \leq p)$, $Y = (y_1, y_2, \ldots, y_M)$, $y_i = (y_{i1}, y_{i2}, \ldots, y_{ij})^T \in Q^{j\times 1}, i = 1, 2, \ldots, M$. Podemos realizar la transferencia óptima con el Error Cuadrático Medio (MSE); los datos originales se pueden reconstruir como:

$$\hat{X} = WY \tag{11}$$

Para construir una red *feed-forward*. En el campo de valor real,(Luo et al., 2023) ha explicado que una red *feed-forward* de una sola capa basada en GHA se aproxima con éxito a los Componentes Principales (PCs), por lo que definimos el QGHA como:

$$W(n+1) = W(n) + \eta(n)\{X(n)Y^H(n) - W(n)\{LT[Y(n)Y^H(n)]\}\} \tag{12}$$

Donde $W \in Q^{p\times j}(j \leq p), X \in Q^{p\times N}, Y \in Q^{j\times M}$, $\eta(n)$ es la tasa de aprendizaje y puede ser una constante. $LT[\,]$ devuelve la parte triangular inferior de la matriz contenida en su interior. Los datos se pueden reconstruir como:

$$Y(n) = W^H(n)X(n) \tag{13}$$

Resultados y Discusión

Para evaluar el rendimiento del método, se elaboró un algoritmo en Python, utilizando la imagen estándar "Lena" (256×256) y la métrica PSNR presentada por (Luo et al., 2023), que mide la calidad visual de la imagen reconstruida en comparación con la original y viene dada por

$$PSNR = 10\log_{10}\left(\frac{3 \times 255^2 \times w \times h}{\sum_{k=1}^{3}\sum_{i=1}^{w}\sum_{j=1}^{h}[In_k(i,j) - I_k(i,j)]^2}\right) \tag{14}$$

Donde In es la imagen original, I es la imagen reconstruida, w y h son el ancho y la altura de la imagen, respectivamente.

Para las simulaciones se usó una tasa de aprendizaje de $\eta = 0.0001$, en la Figura 1 se ilustra la evolución del error cuadrático medio (MSE) durante las primeras 150 iteraciones, dando a entender que la elección de η es crítica. Una tasa conservadora ($\eta = 0.0001$) garantiza un descenso estable del MSE. En contraste, una tasa más alta ($\eta = 0.002$) acelera la convergencia inicial, pero genera oscilaciones en la vecindad del mínimo.

Siguiendo este camino, al extraer las primeras $K = 8$ componentes principales tras 2000 iteraciones, se alcanzó un PSNR destacado de 25.10 dB, logrando un buen equilibrio entre calidad y compresión.

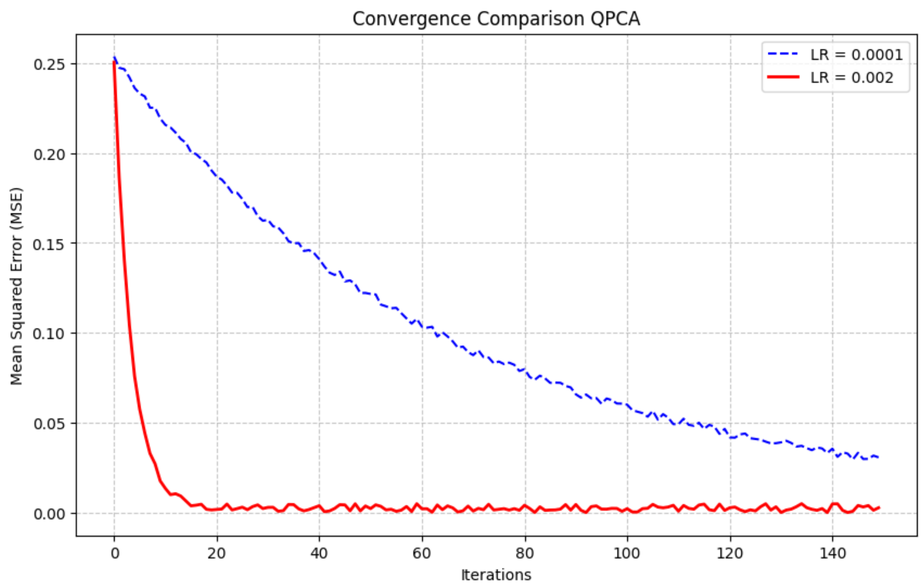

Figure 1. Curva de convergencia del MSE durante el entrenamiento de la QNN.

Además realizamos un análisis de los mapas de residuales, el cual revela el error de reconstrucción de la Red Neuronal de Cuaterniones (QNN). Recordemos que la imagen original se representa como una matriz hipercompleja A de rango m, la imagen reconstruida $\hat{A}$ mediante la red neuronal aproxima los primeros K componentes principales. En consecuencia, el error residual E_{QNN} queda definida por:

$$E_{QNN} = A - \hat{A} = \sum_{i=K+1}^{m} \lambda_i (u_i \times v_i^H) \tag{15}$$

Dado que el algoritmo extrae y ordena las proyecciones maximizando la varianza ($\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_R$), los vectores propios asociados a los valores propios λ_i descartados ($i > K$) corresponden invariablemente a los gradientes de alta frecuencia. Por lo tanto, la red no pierde información al azar, sino que omite de manera determinista los detalles más "finos".

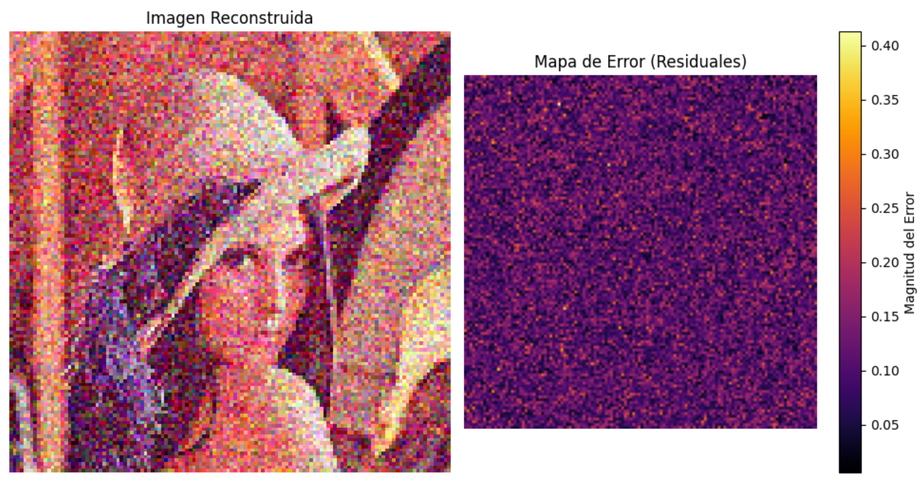

Figure 2. Comparativa visual entre la imagen reconstruida y el mapa de calor de los residuales absolutos.

Conclusiones

- Al tratar los píxeles RGB como una entidad única, las QNN reducen la redundancia de parámetros, optimizando el costo computacional al procesar grandes volúmenes de datos visuales.
- El álgebra de cuaterniones procesa los canales RGB de forma conjunta, evitando las incoherencias cromáticas producto del uso de redes tradicionales.

Referencias

[1] D García-Retuerta, R Casado-Vara, A Martin-del Rey, F De la Prieta, J Prieto, and J. M Corchado. Quaternion neural networks: State-of-the-art and research challenges. *International Conference on Intelligent Data Engineering and Automated Learning*, pages 456–467, 2020.

[2] Lincong Luo, Hao Feng, and Lijun Ding. Color image compression based on quaternion neural network principal component analysis. *2010 International Conference on Multimedia Technology*, pages 1–4, 2010.